

Scrum: Backlog produktu

System przetwarzania danych oparty na pojedynczym oraz współpracujących modelach LLM

1. O projekcie i produkcji

1.1. Krótki opis projektu

Projekt dotyczy stworzenia i badania efektywności systemu przetwarzania danych opartego na współpracujących modelach LLM (tzw. „Hive Mind”). Jego głównym celem jest sprawdzenie, czy grupa współdziałających agentów jest w stanie rozwiązywać złożone problemy matematyczne i logiczne lepiej niż pojedynczy model. Docelowymi odbiorcami są deweloperzy i badacze zajmujący się orkiestracją modeli językowych.

1.2. Krótki opis produktu

Produktem jest system „Hive Mind”, składający się z: agenta badawczego (Researcher), wyspecjalizowanych kalkulatorów (Calculators), agenta oceny wyników (Evaluator). System ma umożliwiać wspólne rozwiązywanie problemów przez wiele agentów oraz ocenę, czy takie podejście daje bardziej wiarygodne i dokładniejsze wyniki niż pojedynczy model — bez potrzeby jego dodatkowego trenowania.

2. Persony użytkowników

2.1. Persona 1: Programista oprogramowania LLM

2.1.1. Podstawowe informacje:

- Michał Kowalski,
- 29 lat,
- Inżynier informatyki,
- Programista systemów AI w firmie technologicznej.

Na co dzień zajmuje się projektowaniem i wdrażaniem aplikacji wykorzystujących modele językowe (LLM), takich jak chatboty narzędzia wspierające analizę danych. Odpowiada za integrację modeli z backendem oraz zapewnienie jakości i stabilności generowanych odpowiedzi w środowisku produkcyjnym.

2.1.2. Używane technologie

Michał pracuje głównie w języku Python, korzystając z bibliotek i frameworków związanych z AI. Używa również API modeli językowych (np. lokalnych modeli przez Ollama lub rozwiązań chmurowych), a także standardowych technologii backendowych (REST API, Docker). W swojej pracy wykorzystuje środowiska eksperymentalne do testowania jakości odpowiedzi modeli oraz narzędzia do monitorowania ich działania.

2.1.3. Problemy:

- pojedyncze modele często generują niepewne lub błędne odpowiedzi,
- fine-tuning jest kosztowny i czasochłonny,
- trudno ocenić jakość odpowiedzi bez dodatkowej warstwy walidacji.

2.1.4. Potrzeby:

- zwiększenie dokładności odpowiedzi modeli LLM,
- ograniczenie halucynacji bez kosztownego trenowania,
- wdrażanie bardziej niezawodnych systemów AI w produkcji.

2.2. Persona 2: Badacz AI

2.2.1. Podstawowe informacje:

- Anna Wiśniewska,
- 32 lat,
- Magister inżynier w zakresie sztucznej inteligencji i uczenia maszynowego,
- Doktorantka zajmująca się systemami wieloagentowymi.

Na co dzień prowadzi badania nad współpracą modeli językowych oraz mechanizmami koordynacji agentów w systemach AI. Zajmuje się projektowaniem eksperymentów, analizą wyników oraz publikowaniem artykułów naukowych. Jej praca obejmuje zarówno aspekty teoretyczne (modele współpracy), jak i praktyczne implementacje systemów eksperymentalnych.

2.2.2. Używane technologie

Anna wykorzystuje głównie język Python oraz narzędzia do analizy danych i eksperymentów, takie jak Jupyter Notebook, NumPy, Pandas i biblioteki do wizualizacji (np. Matplotlib). Pracuje z frameworkami do budowy systemów opartych na LLM (np. LangChain), korzysta z modeli dostępnych lokalnie i przez API, a także używa środowisk obliczeniowych (np. GPU, Docker) do przeprowadzania eksperymentów.

2.2.3. Problemy:

- trudność w porównywaniu wyników między różnymi podejściami,
- konieczność budowania środowisk eksperymentalnych od zera.

2.2.4. Potrzeby:

- badanie efektywności współpracy modeli LLM,
- publikacje naukowe dotyczące orkiestracji agentów,
- eksperymenty z różnymi strategiami wnioskowania.

3. Scenariusz użycia produktu

3.1. Scenariusz 1: Wdrożenie systemu zwiększającego jakość odpowiedzi

Michał pracuje nad wdrożeniem systemu opartego na modelu językowym, który odpowiada na zapytania użytkowników w aplikacji biznesowej. Zauważa, że pojedynczy model LLM generuje czasami niespójne lub niepoprawne odpowiedzi, co obniża wiarygodność całego rozwiązania.

Aby poprawić jakość wyników bez konieczności kosztownego trenowania modelu, Michał decyduje się na wykorzystanie systemu „Hive Mind”. Integruje go jako dodatkową warstwę wnioskowania w swojej aplikacji.

W pierwszym kroku zapytanie użytkownika trafia do agenta typu Researcher, który analizuje problem i generuje wstępne podejścia do jego rozwiązania. Następnie zadanie jest przekazywane do wyspecjalizowanych agentów typu Calculators, którzy wykonują konkretne operacje (np. obliczenia, logiczne przekształcenia lub alternatywne ścieżki rozumowania). W kolejnym etapie wszystkie

uzyskane wyniki trafiają do agenta Evaluator, który porównuje odpowiedzi, ocenia ich spójność i wybiera najbardziej wiarygodną wersję.

Po wdrożeniu rozwiązania zauważa znaczącą poprawę jakości odpowiedzi oraz redukcję halucynacji. System staje się bardziej stabilny i przewidywalny, co pozwala na jego bezpieczne wykorzystanie w środowisku produkcyjnym.

3.2. Scenariusz 2: Eksperyment badawczy nad efektywnością agentów

Anna prowadzi badania nad efektywnością współpracy agentów w systemach opartych na modelach językowych. Jej celem jest porównanie jakości odpowiedzi generowanych przez pojedynczy model LLM oraz przez system wieloagentowy typu „Hive Mind”.

Korzystając z gotowego systemu, Anna przygotowuje zestaw złożonych problemów matematycznych i logicznych, które będą użyte jako benchmark. Następnie konfiguruje środowisko eksperymentalne, definiując role agentów (Researcher, Calculators, Evaluator) oraz parametry działania, takie jak liczba iteracji czy sposób oceny wyników. Uruchamia testy dla pojedynczego modelu LLM i systemu „Hive Mind”. System automatycznie zbiera i porównuje wyniki.

Na podstawie uzyskanych danych Anna formułuje wnioski dotyczące skuteczności kolektywnego wnioskowania oraz przygotowuje publikację naukową. System znacząco skraca czas potrzebny na przygotowanie eksperymentów i eliminuje konieczność budowania całej infrastruktury od podstaw.

4. Backlog produktu

Nr.	Tytuł	Pri.	Oszac.
1	Implementacja podstawowego systemu agentowego	3	8 SP
2	Integracja z lokalnym modelem LLM (Ollama)	3	5 SP
3	Implementacja agenta Researcher	3	5 SP
4	Implementacja agentów Calculators	3	8 SP
5	Implementacja agenta Evaluator	3	5 SP
6	Stworzenie potoku przetwarzania (Research → Calculate → Evaluate)	3	8 SP
7	Mechanizm iteracyjnej poprawy wyników	3	8 SP
8	Konfigurowalna liczba iteracji (CALCULATION_RUNS)	2	3 SP
9	Testowanie na problemach matematycznych i logicznych	2	5 SP
10	Porównanie z pojedynczym modelem LLM	2	5 SP
11	Zbieranie i zapisywanie wyników eksperymentów	2	2 SP
12	Logging i monitoring działania systemu	1	3 SP
13	Możliwość łatwej rozbudowy o nowe typy agentów	1	5 SP
14	Wizualizacja procesu wnioskowania	1	5 SP

5. Kryteria akceptacji

 Dodaj epic /  SCRUM-1



Implementacja podstawowego systemu agentowego



Idea ▾



⌘ Popraw typ zgłoszenia Feature

▼ Opis

Kryteria akceptacji:

- System umożliwia uruchomienie co najmniej dwóch współpracujących agentów LLM
- Agenci mogą wymieniać między sobą dane (np. tekst wejściowy/wyjściowy)
- System przyjmuje zapytanie wejściowe i przekazuje je do agentów
- Wynik działania systemu jest zwracany użytkownikowi
- Kod jest modularny (oddzielne komponenty agentów)
- System uruchamia się lokalnie bez błędów

 Dodaj epic /  SCRUM-5



Implementacja agenta Evaluator



Idea ▾



⌘ Popraw typ zgłoszenia Feature

▼ Opis

Kryteria akceptacji:

- Agent przyjmuje wiele odpowiedzi wygenerowanych przez inne agenty
- Agent porównuje odpowiedzi pod względem poprawności i spójności
- Agent wybiera jedną „najlepszą” odpowiedź lub generuje ocenę rankingową
- Wynik ewaluacji jest przekazywany dalej w systemie
- Możliwe jest łatwe dostosowanie kryteriów oceny (np. promptem)
- Działanie agenta jest powtarzalne dla tych samych danych wejściowych

 Dodaj epic /  SCRUM-6

Stworzenie potoku przetwarzania (Research → Calculate → Evaluate)



Idea ▾



✖ Popraw typ zgłoszenia Feature

▼ Opis

Kryteria akceptacji:

- Dane wejściowe przechodzą przez wszystkie etapy: Researcher → Calculators → Evaluator
- Każdy etap otrzymuje poprawne dane z poprzedniego kroku
- System obsługuje błędy (np. brak odpowiedzi z jednego z agentów)
- Możliwe jest uruchomienie całego pipeline jednym wywołaniem
- Wynik końcowy jest spójny i pochodzi z etapu ewaluacji
- Pipeline działa dla różnych typów problemów (matematyczne, logiczne)

 Dodaj epic /  SCRUM-10

Porównanie z pojedynczym modelem LLM



Idea ▾



✖ Popraw typ zgłoszenia Feature

▼ Opis

Kryteria akceptacji:

- System umożliwia uruchomienie tego samego zapytania dla:
 - pojedynczego modelu LLM
 - systemu Hive Mind
- Wyniki obu podejść są zapisywane
- Możliwe jest porównanie wyników (np. tekstowo lub punktowo)
- System umożliwia analizę różnic między odpowiedziami
- Wyniki mogą być wykorzystane do dalszej analizy (np. eksport/logi)

6. Definicja ukończenia

- Napisano kod
- Napisano testy
- Kod przetestowano i poprawiono błędy
- Wykonano testy integracyjne z Przyrostem
- Kod i testy umieszczono w repozytorium
- Zaktualizowano backlog sprintu